Coordinate Transformation Tutorial
From Theory to Python Implementation

Dr. Weisong Wen, Assistant Professor
AAE4203 - Mathematical Foundations for Guidance and Navigation

July 25, 2025

1 Introduction

Coordinate transformations are fundamental to navigation and guidance systems. This tutorial covers the theoretical foundations and practical Python implementations of major coordinate systems used in navigation: ECEF, LLA, NED, and Body Frame.

1.1 Learning Objectives

- Understand coordinate system mathematical principles
- Implement transformation algorithms in Python
- Apply transformations to real-world navigation problems
- Master Euler angles and quaternion representations
- Practice manual calculations for better understanding

2 Setup and WGS-84 Parameters

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial.transform import Rotation as R
import math

# WGS-84 Ellipsoid Parameters
class WGS84:
    a = 6378137.0          # Semi-major axis (m)
    b = 6356752.314245     # Semi-minor axis (m)
    f = 1/298.257223563    # Flattening
    e2 = 6.69437999014e-3  # First eccentricity squared

# Helper functions
def deg2rad(degrees):
    return np.deg2rad(degrees)

def rad2deg(radians):
    return np.rad2deg(radians)

print(f"WGS-84 Parameters:")
print(f"Semi-major axis: {WGS84.a:,.0f} m")
print(f"Semi-minor axis: {WGS84.b:,.3f} m")
print(f"Flattening: 1/{1/WGS84.f:.0f}")
```

Listing 1: Setup and WGS-84 parameters

3 ECEF LLA Transformations

3.1 Theory

The Earth-Centered Earth-Fixed (ECEF) system is Cartesian with origin at Earth's center. Latitude-Longitude-Altitude (LLA) uses geodetic coordinates on the WGS-84 ellipsoid.

3.2 LLA to ECEF Conversion

$$N = \frac{a}{\sqrt{1 - e^2 \sin^2 \phi}} \tag{1}$$

$$x = (N + h) \cos \phi \cos \lambda \tag{2}$$

$$y = (N + h) \cos \phi \sin \lambda \tag{3}$$

$$z = (N(1 - e^2) + h) \sin \phi \tag{4}$$

Practice Problem 1: Manual LLA to ECEF Conversion

Convert the coordinates of a point in Singapore to ECEF coordinates.

Given:

- Latitude: $\phi = 1.3521$ N
- Longitude: $\lambda = 103.8198$ E
- Altitude: $h = 15$ m above ellipsoid
- WGS-84 parameters: $a = 6378137$ m, $e^2 = 0.00669438$

Calculate by hand:

1. Convert angles to radians
2. Calculate radius of curvature N
3. Calculate ECEF coordinates (x, y, z)

Solution: Step 1: Convert to radians

$$\phi = 1.3521 \times \frac{\pi}{180} = 0.023598 \text{ rad} \tag{5}$$

$$\lambda = 103.8198 \times \frac{\pi}{180} = 1.812416 \text{ rad} \tag{6}$$

Step 2: Calculate trigonometric values

$$\sin \phi = \sin(0.023598) = 0.023596 \tag{7}$$

$$\cos \phi = \cos(0.023598) = 0.999721 \tag{8}$$

$$\sin \lambda = \sin(1.812416) = 0.975584 \tag{9}$$

$$\cos \lambda = \cos(1.812416) = -0.219630 \tag{10}$$

Step 3: Calculate radius of curvature

$$N = \frac{a}{\sqrt{1 - e^2 \sin^2 \phi}} \tag{11}$$

$$= \frac{6378137}{\sqrt{1 - 0.00669438 \times (0.023596)^2}} \tag{12}$$

$$= \frac{6378137}{\sqrt{1 - 0.00669438 \times 0.000557}} \tag{13}$$

$$= \frac{6378137}{\sqrt{0.999996}} \tag{14}$$

$$= 6378139.2 \text{ m} \tag{15}$$

Step 4: Calculate ECEF coordinates

$$
\begin{aligned}
x &= (N+h)\cos\phi\cos\lambda && (16)\\
&= (6378139.2+15)\times 0.999721\times(-0.219630) && (17)\\
&= 6378154.2\times(-0.219569) && (18)\\
&= -1401095.7\ \text{m} && (19)\\
y &= (N+h)\cos\phi\sin\lambda && (20)\\
&= 6378154.2\times 0.999721\times 0.975584 && (21)\\
&= 6221169.4\ \text{m} && (22)\\
z &= (N(1-e^2)+h)\sin\phi && (23)\\
&= (6378139.2\times 0.99331+15)\times 0.023596 && (24)\\
&= 6335459.2\times 0.023596 && (25)\\
&= 149512.5\ \text{m} && (26)
\end{aligned}
$$

Final Answer: ECEF $= [-1401095.7, 6221169.4, 149512.5]$ m

```python
def lla_to_ecef(lat_deg, lon_deg, alt_m):
    """Convert LLA to ECEF coordinates"""
    lat_rad = deg2rad(lat_deg)
    lon_rad = deg2rad(lon_deg)

    sin_lat = np.sin(lat_rad)
    cos_lat = np.cos(lat_rad)
    cos_lon = np.cos(lon_rad)
    sin_lon = np.sin(lon_rad)

    # Radius of curvature in prime vertical
    N = WGS84.a / np.sqrt(1 - WGS84.e2 * sin_lat**2)

    # ECEF coordinates
    x = (N + alt_m) * cos_lat * cos_lon
    y = (N + alt_m) * cos_lat * sin_lon
    z = (N * (1 - WGS84.e2) + alt_m) * sin_lat

    return np.array([x, y, z])

def ecef_to_lla(ecef_coords, max_iterations=10):
    """Convert ECEF to LLA using iterative method"""
    x, y, z = ecef_coords

    # Longitude calculation
    lon_rad = np.arctan2(y, x)

    # Iterative latitude/altitude calculation
    p = np.sqrt(x**2 + y**2)
    lat_rad = np.arctan2(z, p * (1 - WGS84.e2))

    for i in range(max_iterations):
        sin_lat = np.sin(lat_rad)
        N = WGS84.a / np.sqrt(1 - WGS84.e2 * sin_lat**2)
        alt_m = p / np.cos(lat_rad) - N
        lat_rad_new = np.arctan2(z, p * (1 - WGS84.e2 * N / (N + alt_m)))

        if abs(lat_rad_new - lat_rad) < 1e-12:
            break
        lat_rad = lat_rad_new

    return np.array([rad2deg(lat_rad), rad2deg(lon_rad), alt_m])

# Verify the manual calculation
singapore_lla = [1.3521, 103.8198, 15.0]
singapore_ecef = lla_to_ecef(*singapore_lla)
print(f"Singapore ECEF: {singapore_ecef}")
```

Listing 2: LLA to ECEF implementation

4 ECEF NED Transformations

4.1 Theory

The North-East-Down (NED) system is a local tangent plane coordinate system. The rotation matrix from ECEF to NED is:

$$\mathbf{R}_{ECEF}^{NED} = \begin{bmatrix} -\sin\phi\cos\lambda & -\sin\phi\sin\lambda & \cos\phi \\ -\sin\lambda & \cos\lambda & 0 \\ -\cos\phi\cos\lambda & -\cos\phi\sin\lambda & -\sin\phi \end{bmatrix} \tag{27}$$

Practice Problem 2: Manual ECEF to NED Rotation Matrix

Calculate the rotation matrix from ECEF to NED for PolyU location.

Given:

- PolyU coordinates: $\phi = 22.303$ N, $\lambda = 114.180$ E

Calculate by hand:

1. Convert angles to radians
2. Calculate trigonometric values
3. Construct the 3×3 rotation matrix

Solution: Step 1: Convert to radians

$$\phi = 22.303 \times \frac{\pi}{180} = 0.3892 \text{ rad} \tag{28}$$

$$\lambda = 114.180 \times \frac{\pi}{180} = 1.9927 \text{ rad} \tag{29}$$

Step 2: Calculate trigonometric values

$$\sin\phi = \sin(0.3892) = 0.3789 \tag{30}$$

$$\cos\phi = \cos(0.3892) = 0.9255 \tag{31}$$

$$\sin\lambda = \sin(1.9927) = 0.9105 \tag{32}$$

$$\cos\lambda = \cos(1.9927) = -0.4135 \tag{33}$$

Step 3: Construct rotation matrix

$$\mathbf{R}_{ECEF}^{NED} = \begin{bmatrix} -\sin\phi\cos\lambda & -\sin\phi\sin\lambda & \cos\phi \\ -\sin\lambda & \cos\lambda & 0 \\ -\cos\phi\cos\lambda & -\cos\phi\sin\lambda & -\sin\phi \end{bmatrix} \tag{34}$$

$$= \begin{bmatrix} -0.3789 \times (-0.4135) & -0.3789 \times 0.9105 & 0.9255 \\ -0.9105 & -0.4135 & 0 \\ -0.9255 \times (-0.4135) & -0.9255 \times 0.9105 & -0.3789 \end{bmatrix} \tag{35}$$

$$= \begin{bmatrix} 0.1567 & -0.3451 & 0.9255 \\ -0.9105 & -0.4135 & 0 \\ 0.3828 & -0.8427 & -0.3789 \end{bmatrix} \tag{36}$$

Verification: Check that $\mathbf{R}\mathbf{R}^T = \mathbf{I}$ and $\det(\mathbf{R}) = 1$.

Practice Problem 3: Manual NED Coordinate Transformation

A point is located at coordinates [100, -50, 20] meters in ECEF relative to PolyU. Calculate its NED coordinates using the rotation matrix from Practice Problem 2.

Given:

- Relative ECEF vector: $\mathbf{p}_{rel} = [100, -50, 20]^T$ m
- Rotation matrix from Practice Problem 2

Calculate: $\mathbf{p}_{NED} = \mathbf{R}_{ECEF}^{NED}\mathbf{p}_{rel}$

Solution: Matrix multiplication:

$$\mathbf{p}_{NED} = \begin{bmatrix} 0.1567 & -0.3451 & 0.9255 \\ -0.9105 & -0.4135 & 0 \\ 0.3828 & -0.8427 & -0.3789 \end{bmatrix} \begin{bmatrix} 100 \\ -50 \\ 20 \end{bmatrix} \tag{37}$$

$$= \begin{bmatrix} 0.1567 \times 100 + (-0.3451) \times (-50) + 0.9255 \times 20 \\ -0.9105 \times 100 + (-0.4135) \times (-50) + 0 \times 20 \\ 0.3828 \times 100 + (-0.8427) \times (-50) + (-0.3789) \times 20 \end{bmatrix} \tag{38}$$

$$= \begin{bmatrix} 15.67 + 17.255 + 18.51 \\ -91.05 + 20.675 + 0 \\ 38.28 + 42.135 - 7.578 \end{bmatrix} \tag{39}$$

$$= \begin{bmatrix} 51.435 \\ -70.375 \\ 72.837 \end{bmatrix} \text{ m} \tag{40}$$

Final Answer: NED coordinates = [51.4, -70.4, 72.8] m

- North: 51.4 m
- East: -70.4 m (70.4 m West)
- Down: 72.8 m

```python
def ecef_to_ned_matrix(ref_lat_deg, ref_lon_deg):
    """Calculate rotation matrix from ECEF to NED"""
    lat_rad = deg2rad(ref_lat_deg)
    lon_rad = deg2rad(ref_lon_deg)

    sin_lat, cos_lat = np.sin(lat_rad), np.cos(lat_rad)
    sin_lon, cos_lon = np.sin(lon_rad), np.cos(lon_rad)

    R = np.array([
        [-sin_lat * cos_lon, -sin_lat * sin_lon,  cos_lat],
        [-sin_lon,            cos_lon,            0      ],
        [-cos_lat * cos_lon, -cos_lat * sin_lon, -sin_lat]
    ])
    return R

def ecef_to_ned(ecef_point, ecef_ref, ref_lat_deg, ref_lon_deg):
    """Convert ECEF to NED relative to reference point"""
    ecef_rel = np.array(ecef_point) - np.array(ecef_ref)
    R_ecef_to_ned = ecef_to_ned_matrix(ref_lat_deg, ref_lon_deg)
    return R_ecef_to_ned @ ecef_rel

# Verify the manual calculations
polyu_coords = [22.303, 114.180]
R_manual = ecef_to_ned_matrix(*polyu_coords)
print(f"Rotation matrix:\n{R_manual}")

# Verify Practice Problem 3
ecef_relative = np.array([100, -50, 20])
ned_result = R_manual @ ecef_relative
```

```
print(f"NED coordinates: {ned_result}")
```

Listing 3: ECEF NED transformations

5 Body Frame and Euler Angles

5.1 Theory

The body frame is fixed to a vehicle with axes: X (forward), Y (right), Z (down). Orientation is described using Euler angles: roll (), pitch (), yaw () in the 3-2-1 sequence.

Practice Problem 4: Manual Euler Angle Rotation Matrices

Calculate the individual rotation matrices and combined rotation matrix for an aircraft with the following attitude:

Given:

- Roll (): 15°
- Pitch (): 10°
- Yaw (): 30°

Calculate by hand:

1. Individual rotation matrices $R_x()$, $R_y()$, $R_z()$
2. Combined rotation matrix $R = R_x()R_y()R_z()$ (3-2-1 sequence)

Solution: Step 1: Convert to radians

$$\phi = 15 = 0.2618 \text{ rad} \tag{41}$$
$$\theta = 10 = 0.1745 \text{ rad} \tag{42}$$
$$\psi = 30 = 0.5236 \text{ rad} \tag{43}$$

Step 2: Calculate trigonometric values

$$\cos(15) = 0.9659, \quad \sin(15) = 0.2588 \tag{44}$$
$$\cos(10) = 0.9848, \quad \sin(10) = 0.1736 \tag{45}$$
$$\cos(30) = 0.8660, \quad \sin(30) = 0.5000 \tag{46}$$

Step 3: Individual rotation matrices

Roll rotation about X-axis:

$$R_x(15) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.9659 & 0.2588 \\ 0 & -0.2588 & 0.9659 \end{bmatrix} \tag{47}$$

Pitch rotation about Y-axis:

$$R_y(10) = \begin{bmatrix} 0.9848 & 0 & -0.1736 \\ 0 & 1 & 0 \\ 0.1736 & 0 & 0.9848 \end{bmatrix} \tag{48}$$

Yaw rotation about Z-axis:

$$R_z(30) = \begin{bmatrix} 0.8660 & 0.5000 & 0 \\ -0.5000 & 0.8660 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{49}$$

Step 4: Combined rotation matrix (multiply right to left)

$$R = R_x()R_y()R_z() \tag{50}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.9659 & 0.2588 \\ 0 & -0.2588 & 0.9659 \end{bmatrix} \begin{bmatrix} 0.9848 & 0 & -0.1736 \\ 0 & 1 & 0 \\ 0.1736 & 0 & 0.9848 \end{bmatrix} \begin{bmatrix} 0.8660 & 0.5000 & 0 \\ -0.5000 & 0.8660 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{51}$$

First, $R_y \times R_z$:

$$R_y R_z = \begin{bmatrix} 0.8527 & 0.4924 & -0.1736 \\ -0.5000 & 0.8660 & 0 \\ 0.1503 & 0.0868 & 0.9848 \end{bmatrix} \tag{52}$$

Then, $R_x \times (R_y R_z)$:

$$R = \begin{bmatrix} 0.8527 & 0.4924 & -0.1736 \\ -0.5224 & 0.8365 & 0.1629 \\ 0.1060 & 0.2198 & 0.9696 \end{bmatrix} \tag{53}$$

Practice Problem 5: Manual Body to NED Vector Transformation

A velocity vector in the aircraft body frame is $\mathbf{v}_{body} = [20, 5, -2]^T$ m/s (forward, right, down). Using the rotation matrix from Practice Problem 4, calculate the velocity in NED frame.

Note: Body to NED transformation uses $\mathbf{R}^T$ (transpose of NED to Body matrix).

Solution: Step 1: Transpose the rotation matrix

$$R_{body}^{NED} = R^T = \begin{bmatrix} 0.8527 & -0.5224 & 0.1060 \\ 0.4924 & 0.8365 & 0.2198 \\ -0.1736 & 0.1629 & 0.9696 \end{bmatrix} \tag{54}$$

Step 2: Transform the velocity vector

$$\mathbf{v}_{NED} = R^T \mathbf{v}_{body} \tag{55}$$

$$= \begin{bmatrix} 0.8527 & -0.5224 & 0.1060 \\ 0.4924 & 0.8365 & 0.2198 \\ -0.1736 & 0.1629 & 0.9696 \end{bmatrix} \begin{bmatrix} 20 \\ 5 \\ -2 \end{bmatrix} \tag{56}$$

$$= \begin{bmatrix} 0.8527 \times 20 + (-0.5224) \times 5 + 0.1060 \times (-2) \\ 0.4924 \times 20 + 0.8365 \times 5 + 0.2198 \times (-2) \\ -0.1736 \times 20 + 0.1629 \times 5 + 0.9696 \times (-2) \end{bmatrix} \tag{57}$$

$$= \begin{bmatrix} 17.054 - 2.612 - 0.212 \\ 9.848 + 4.183 - 0.440 \\ -3.472 + 0.815 - 1.939 \end{bmatrix} \tag{58}$$

$$= \begin{bmatrix} 14.23 \\ 13.59 \\ -4.60 \end{bmatrix} \text{ m/s} \tag{59}$$

Final Answer: NED velocity = [14.23, 13.59, -4.60] m/s

- North velocity: 14.23 m/s
- East velocity: 13.59 m/s

- Down velocity: -4.60 m/s (upward motion)

Ground speed: $\sqrt{14.23^2 + 13.59^2} = 19.73$ m/s
Track angle: $\tan^{-1}(13.59/14.23) = 43.7$ from North

```python
def euler_to_rotation_matrix(roll_deg, pitch_deg, yaw_deg):
    """Convert Euler angles to rotation matrix (3-2-1 sequence)"""
    roll, pitch, yaw = deg2rad([roll_deg, pitch_deg, yaw_deg])

    # Individual rotation matrices
    Rx = np.array([[1, 0, 0],
                   [0, np.cos(roll), np.sin(roll)],
                   [0, -np.sin(roll), np.cos(roll)]])

    Ry = np.array([[np.cos(pitch), 0, -np.sin(pitch)],
                   [0, 1, 0],
                   [np.sin(pitch), 0, np.cos(pitch)]])

    Rz = np.array([[np.cos(yaw), np.sin(yaw), 0],
                   [-np.sin(yaw), np.cos(yaw), 0],
                   [0, 0, 1]])

    return Rx @ Ry @ Rz  # 3-2-1 sequence

def body_to_ned_transform(vector_body, roll_deg, pitch_deg, yaw_deg):
    """Transform vector from body frame to NED frame"""
    R_ned_to_body = euler_to_rotation_matrix(roll_deg, pitch_deg, yaw_deg)
    R_body_to_ned = R_ned_to_body.T
    return R_body_to_ned @ np.array(vector_body)

# Verify Practice Problems 4 and 5
R_manual = euler_to_rotation_matrix(15, 10, 30)
print(f"Combined rotation matrix:\n{R_manual}")

v_body = np.array([20, 5, -2])
v_ned = body_to_ned_transform(v_body, 15, 10, 30)
print(f"NED velocity: {v_ned}")

ground_speed = np.sqrt(v_ned[0]**2 + v_ned[1]**2)
track_angle = rad2deg(np.arctan2(v_ned[1], v_ned[0]))
print(f"Ground speed: {ground_speed:.2f} m/s")
print(f"Track angle: {track_angle:.1f}   from North")
```

Listing 4: Euler angles and body frame transformations

6 Quaternions

6.1 Theory

Quaternions provide singularity-free rotation representation using four parameters:

$$\mathbf{q} = [q_0, q_1, q_2, q_3] = [\cos\frac{\theta}{2}, \mathbf{u}\sin\frac{\theta}{2}] \tag{60}$$

Practice Problem 6: Manual Quaternion from Euler Angles

Convert the Euler angles from Practice Problem 4 to quaternion representation.
Given: Roll = 15°, Pitch = 10°, Yaw = 30°
Formula for 3-2-1 sequence:

$$q_0 = \cos\frac{\phi}{2}\cos\frac{\theta}{2}\cos\frac{\psi}{2} + \sin\frac{\phi}{2}\sin\frac{\theta}{2}\sin\frac{\psi}{2} \tag{61}$$

$$q_1 = \sin\frac{\phi}{2}\cos\frac{\theta}{2}\cos\frac{\psi}{2} - \cos\frac{\phi}{2}\sin\frac{\theta}{2}\sin\frac{\psi}{2} \tag{62}$$

$$q_2 = \cos\frac{\phi}{2}\sin\frac{\theta}{2}\cos\frac{\psi}{2} + \sin\frac{\phi}{2}\cos\frac{\theta}{2}\sin\frac{\psi}{2} \tag{63}$$

$$q_3 = \cos\frac{\phi}{2}\cos\frac{\theta}{2}\sin\frac{\psi}{2} - \sin\frac{\phi}{2}\sin\frac{\theta}{2}\cos\frac{\psi}{2} \tag{64}$$

Solution: Step 1: Calculate half angles

$$\frac{\phi}{2} = \frac{15}{2} = 7.5 = 0.1309 \text{ rad} \tag{65}$$
$$\frac{\theta}{2} = \frac{10}{2} = 5 = 0.0873 \text{ rad} \tag{66}$$
$$\frac{\psi}{2} = \frac{30}{2} = 15 = 0.2618 \text{ rad} \tag{67}$$

Step 2: Calculate trigonometric values

$$\cos(7.5) = 0.9914, \quad \sin(7.5) = 0.1305 \tag{68}$$
$$\cos(5) = 0.9962, \quad \sin(5) = 0.0872 \tag{69}$$
$$\cos(15) = 0.9659, \quad \sin(15) = 0.2588 \tag{70}$$

Step 3: Calculate quaternion components

$$q_0 = 0.9914 \times 0.9962 \times 0.9659 + 0.1305 \times 0.0872 \times 0.2588 \tag{71}$$
$$= 0.9540 + 0.0029 = 0.9569 \tag{72}$$
$$q_1 = 0.1305 \times 0.9962 \times 0.9659 - 0.9914 \times 0.0872 \times 0.2588 \tag{73}$$
$$= 0.1255 - 0.0224 = 0.1031 \tag{74}$$
$$q_2 = 0.9914 \times 0.0872 \times 0.9659 + 0.1305 \times 0.9962 \times 0.2588 \tag{75}$$
$$= 0.0835 + 0.0336 = 0.1171 \tag{76}$$
$$q_3 = 0.9914 \times 0.9962 \times 0.2588 - 0.1305 \times 0.0872 \times 0.9659 \tag{77}$$
$$= 0.2554 - 0.0110 = 0.2444 \tag{78}$$

Step 4: Verify unit quaternion

$$|\mathbf{q}| = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2} \tag{79}$$
$$= \sqrt{0.9569^2 + 0.1031^2 + 0.1171^2 + 0.2444^2} \tag{80}$$
$$= \sqrt{0.9157 + 0.0106 + 0.0137 + 0.0597} \tag{81}$$
$$= \sqrt{0.9997} = 0.9999 \approx 1 \tag{82}$$

Final Answer: $\mathbf{q} = [0.9569, 0.1031, 0.1171, 0.2444]$

Practice Problem 7: Manual Quaternion to Rotation Matrix

Convert the quaternion from Practice Problem 6 to a rotation matrix and verify it matches the result from Practice Problem 4.

Formula:

$$\mathbf{R} = \begin{bmatrix} 1 - 2(q_2^2 + q_3^2) & 2(q_1 q_2 - q_0 q_3) & 2(q_1 q_3 + q_0 q_2) \\ 2(q_1 q_2 + q_0 q_3) & 1 - 2(q_1^2 + q_3^2) & 2(q_2 q_3 - q_0 q_1) \\ 2(q_1 q_3 - q_0 q_2) & 2(q_2 q_3 + q_0 q_1) & 1 - 2(q_1^2 + q_2^2) \end{bmatrix} \tag{83}$$

Solution: Given: $\mathbf{q} = [0.9569, 0.1031, 0.1171, 0.2444]$

Step 1: Calculate squared terms

$$q_0^2 = 0.9157, \quad q_1^2 = 0.0106 \tag{84}$$
$$q_2^2 = 0.0137, \quad q_3^2 = 0.0597 \tag{85}$$

Step 2: Calculate cross products

$$q_1 q_2 = 0.1031 \times 0.1171 = 0.0121 \tag{86}$$
$$q_1 q_3 = 0.1031 \times 0.2444 = 0.0252 \tag{87}$$
$$q_2 q_3 = 0.1171 \times 0.2444 = 0.0286 \tag{88}$$
$$q_0 q_1 = 0.9569 \times 0.1031 = 0.0987 \tag{89}$$
$$q_0 q_2 = 0.9569 \times 0.1171 = 0.1121 \tag{90}$$
$$q_0 q_3 = 0.9569 \times 0.2444 = 0.2339 \tag{91}$$

Step 3: Calculate matrix elements

$$R_{11} = 1 - 2(0.0137 + 0.0597) = 1 - 0.1468 = 0.8532 \tag{92}$$
$$R_{12} = 2(0.0121 - 0.2339) = 2(-0.2218) = -0.4436 \tag{93}$$
$$R_{13} = 2(0.0252 + 0.1121) = 2(0.1373) = 0.2746 \tag{94}$$
$$R_{21} = 2(0.0121 + 0.2339) = 2(0.2460) = 0.4920 \tag{95}$$
$$R_{22} = 1 - 2(0.0106 + 0.0597) = 1 - 0.1406 = 0.8594 \tag{96}$$
$$R_{23} = 2(0.0286 - 0.0987) = 2(-0.0701) = -0.1402 \tag{97}$$
$$R_{31} = 2(0.0252 - 0.1121) = 2(-0.0869) = -0.1738 \tag{98}$$
$$R_{32} = 2(0.0286 + 0.0987) = 2(0.1273) = 0.2546 \tag{99}$$
$$R_{33} = 1 - 2(0.0106 + 0.0137) = 1 - 0.0486 = 0.9514 \tag{100}$$

Final rotation matrix:

$$\mathbf{R} = \begin{bmatrix} 0.8532 & -0.4436 & 0.2746 \\ 0.4920 & 0.8594 & -0.1402 \\ -0.1738 & 0.2546 & 0.9514 \end{bmatrix} \tag{101}$$

Note: This should closely match the result from Practice Problem 4, with small differences due to rounding errors in manual calculations.

```python
class Quaternion:
    def __init__(self, w, x, y, z):
        self.w, self.x, self.y, self.z = w, x, y, z
        self.normalize()

    @classmethod
    def from_euler(cls, roll_deg, pitch_deg, yaw_deg):
        """Create quaternion from Euler angles"""
        roll, pitch, yaw = deg2rad([roll_deg, pitch_deg, yaw_deg]) / 2

        cr, sr = np.cos(roll), np.sin(roll)
        cp, sp = np.cos(pitch), np.sin(pitch)
        cy, sy = np.cos(yaw), np.sin(yaw)

        w = cr * cp * cy + sr * sp * sy
        x = sr * cp * cy - cr * sp * sy
        y = cr * sp * cy + sr * cp * sy
        z = cr * cp * sy - sr * sp * cy

        return cls(w, x, y, z)

    def normalize(self):
        """Normalize to unit quaternion"""
        norm = np.sqrt(self.w**2 + self.x**2 + self.y**2 + self.z**2)
        if norm > 1e-10:
            self.w /= norm
            self.x /= norm
```

```
            self.y /= norm
            self.z /= norm

    def to_rotation_matrix(self):
        """Convert to rotation matrix"""
        w, x, y, z = self.w, self.x, self.y, self.z
        return np.array([
            [1-2*(y**2+z**2), 2*(x*y-w*z), 2*(x*z+w*y)],
            [2*(x*y+w*z), 1-2*(x**2+z**2), 2*(y*z-w*x)],
            [2*(x*z-w*y), 2*(y*z+w*x), 1-2*(x**2+y**2)]
        ])

    def __repr__(self):
        return f"Q({self.w:.4f}, {self.x:.4f}, {self.y:.4f}, {self.z:.4f})"

# Verify Practice Problems 6 and 7
q_manual = Quaternion.from_euler(15, 10, 30)
print(f"Quaternion from Euler: {q_manual}")

R_from_quat = q_manual.to_rotation_matrix()
print(f"Rotation matrix from quaternion:\n{R_from_quat}")

# Compare with Euler matrix
R_from_euler = euler_to_rotation_matrix(15, 10, 30)
diff = np.max(np.abs(R_from_quat - R_from_euler))
print(f"Maximum difference: {diff:.2e}")
```

Listing 5: Quaternion implementation and verification

7 Summary and Additional Practice

Practice Problem 8: Comprehensive Problem

A UAV is flying at coordinates 22.308°N, 114.182°E, 200m altitude with attitude: Roll=5°, Pitch=-3°, Yaw=60°. A target is detected by the onboard camera at bearing 45° right and 10° down relative to the aircraft's heading.

Calculate by hand:

1. Convert UAV position to ECEF coordinates
2. If the target is 500m away, what is its position vector in body frame?
3. Convert this to NED frame relative to the UAV
4. Find the target's ECEF coordinates
5. Convert target ECEF to LLA coordinates

Solution: This is left as an exercise for students to practice the complete transformation chain. The solution involves:

1. Using LLA to ECEF conversion formulas 2. Creating a body frame vector from range and bearing 3. Using the attitude to transform to NED frame 4. Converting NED displacement to ECEF displacement 5. Using iterative ECEF to LLA conversion

Students should work through each step methodically, applying the techniques learned in previous practice problems.

7.1 Key Takeaways from Manual Calculations

Benefits of hand calculations:

- Deeper understanding of transformation mathematics
- Ability to debug automated calculations
- Recognition of reasonable vs unreasonable results

- Understanding of computational complexity and precision requirements

Common sources of error in manual calculations:

- Rounding errors accumulating through multiple steps
- Incorrect angle unit conversions (degrees vs radians)
- Matrix multiplication order errors
- Sign convention mistakes

Best practices:

- Keep more decimal places in intermediate calculations
- Double-check matrix dimensions and multiplication order
- Verify results using inverse transformations
- Use physical intuition to check if results make sense

This tutorial provided both theoretical understanding and practical implementation skills for coordinate transformations essential in navigation and guidance systems. The manual calculations reinforce the mathematical foundations while the Python implementations enable real-world applications.